

SERVICIO NACIONAL DE APRENDIZAJE SENA

Centro: Centro de Diseño e Innovación Tecnológica Industrial

Ficha: 3235906

Programa de Formación: TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE

Nombre del Aprendiz: Laura Barona Saavedra

Nombre del Instructor: Andrés Felipe Parra Martínez

Fecha: 5 de julio 2026

INTRODUCCION

El presente informe describe el proceso de codificación del backend del proyecto Hogar360, una plataforma inmobiliaria desarrollada para facilitar la publicación de inmuebles, la búsqueda de propiedades y la gestión de visitas entre compradores y vendedores.

El desarrollo se realizó utilizando Java 17, Spring Boot y la Arquitectura de Puertos y Adaptadores (Arquitectura Hexagonal), con el propósito de construir un software escalable, mantenible y alineado con los requerimientos funcionales definidos durante el análisis del proyecto.

Durante la implementación se desarrollaron diferentes módulos de negocio, cada uno organizado siguiendo una separación clara de responsabilidades entre la lógica del dominio y los componentes de infraestructura. Asimismo, se aplicaron buenas prácticas de programación, principios SOLID y patrones de diseño que favorecen la reutilización del código y facilitan futuras modificaciones del sistema.

OBJETIVO GENERAL

Describir el proceso de codificación de los módulos que conforman el backend del proyecto Hogar360, evidenciando la implementación de los requerimientos mediante una arquitectura basada en puertos y adaptadores.

OBJETIVOS ESPECÍFICOS

- Implementar la lógica de negocio de acuerdo con los requerimientos funcionales.
- Organizar el código utilizando Arquitectura Hexagonal.
- Implementar servicios REST para la comunicación con el frontend.
- Garantizar la integridad de la información mediante validaciones.
- Desacoplar la lógica del negocio de la infraestructura de persistencia.

ARQUITECTURA IMPLEMENTADA

El backend fue desarrollado siguiendo la Arquitectura Hexagonal, también conocida como Arquitectura de Puertos y Adaptadores.

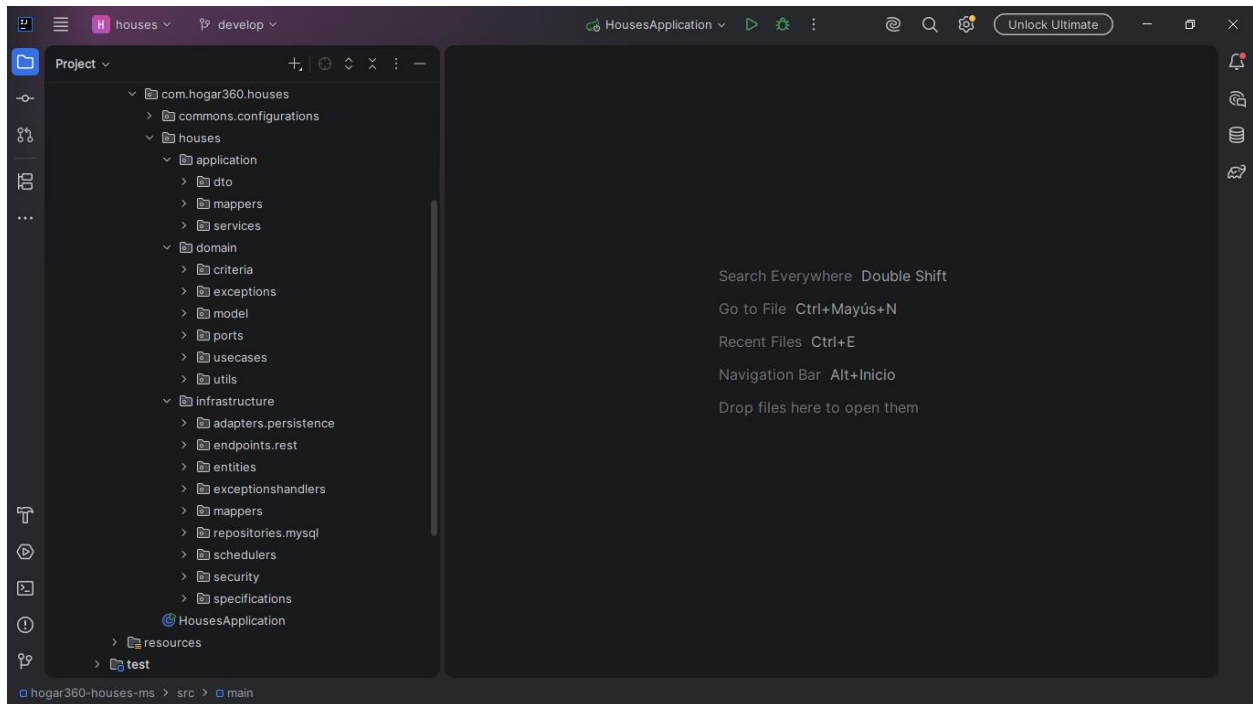
Este modelo permite separar completamente la lógica del negocio de las tecnologías externas, como la base de datos o los servicios web.

La estructura principal del proyecto está conformada por:

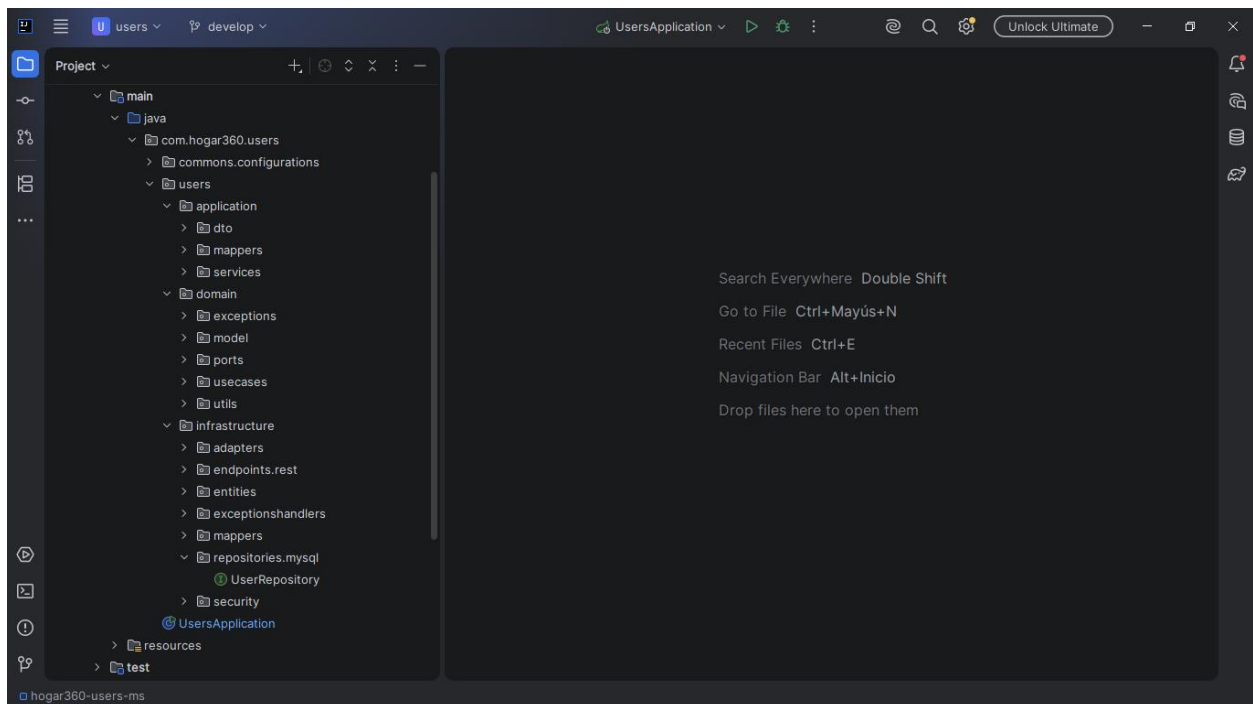
- Dominio
- Casos de uso
- Puertos
- Adaptadores
- Infraestructura

Gracias a esta organización, cualquier cambio en la base de datos o en la interfaz de usuario puede realizarse sin afectar la lógica principal del sistema.

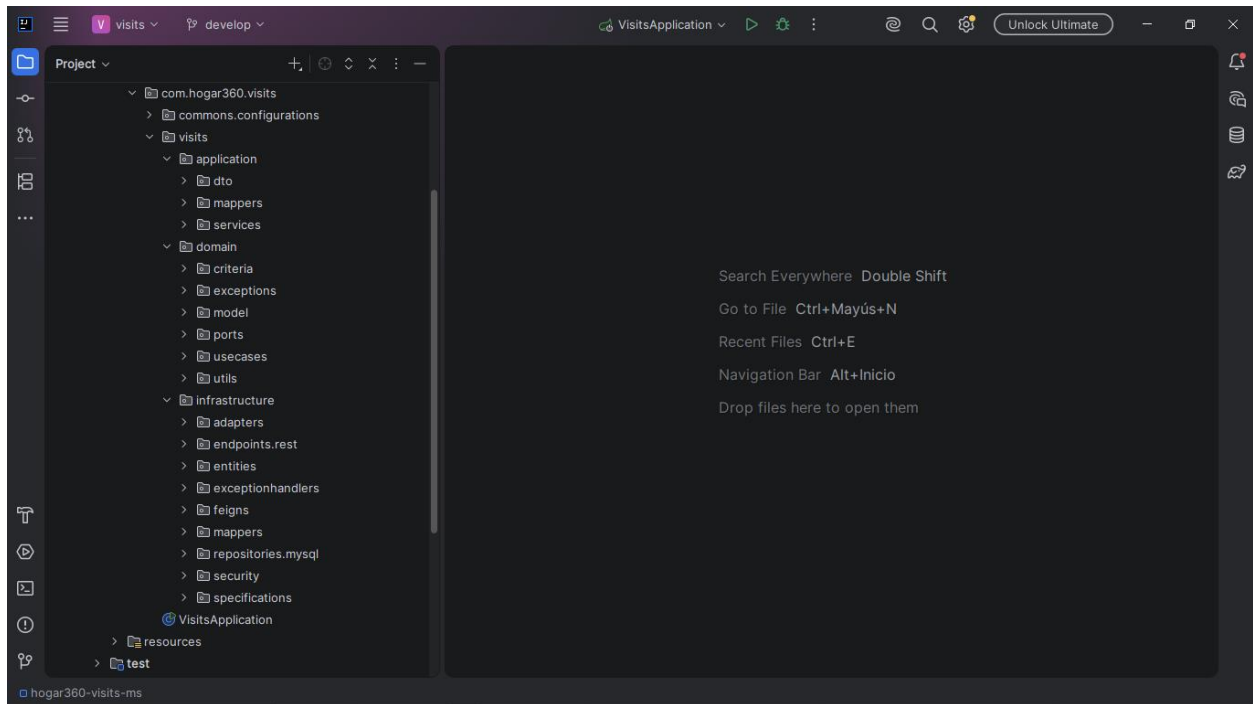
Houses microservice



User microservice



Visits microservice



MODELO DE DOMINIO

El dominio representa el núcleo del sistema y contiene las entidades principales junto con las reglas de negocio.

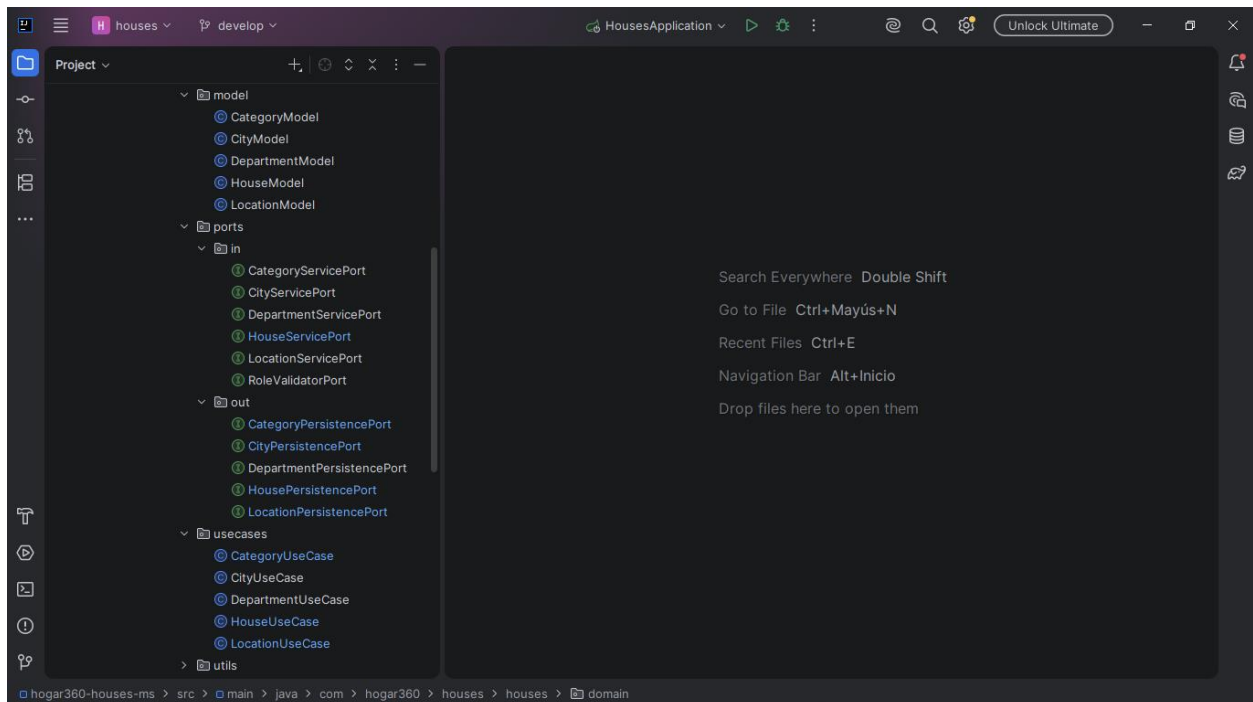
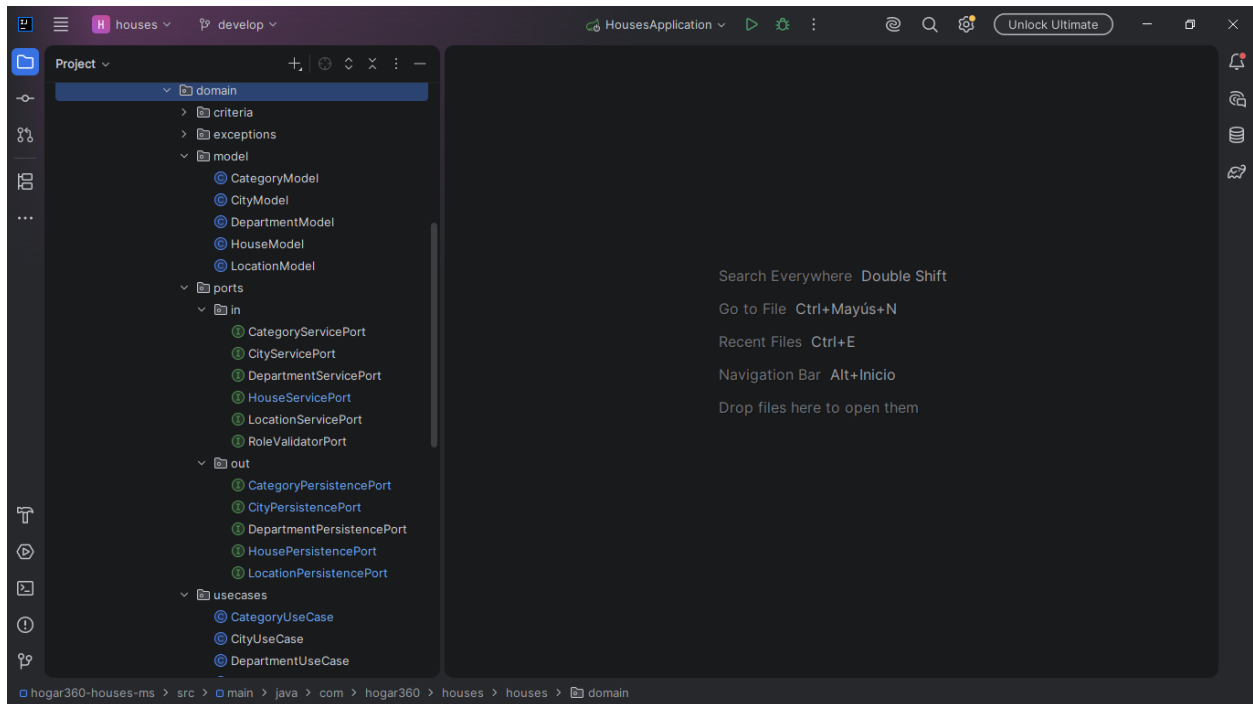
En Hogar360 se implementaron modelos como:

- House
- Category
- Location
- Visit
- Seller

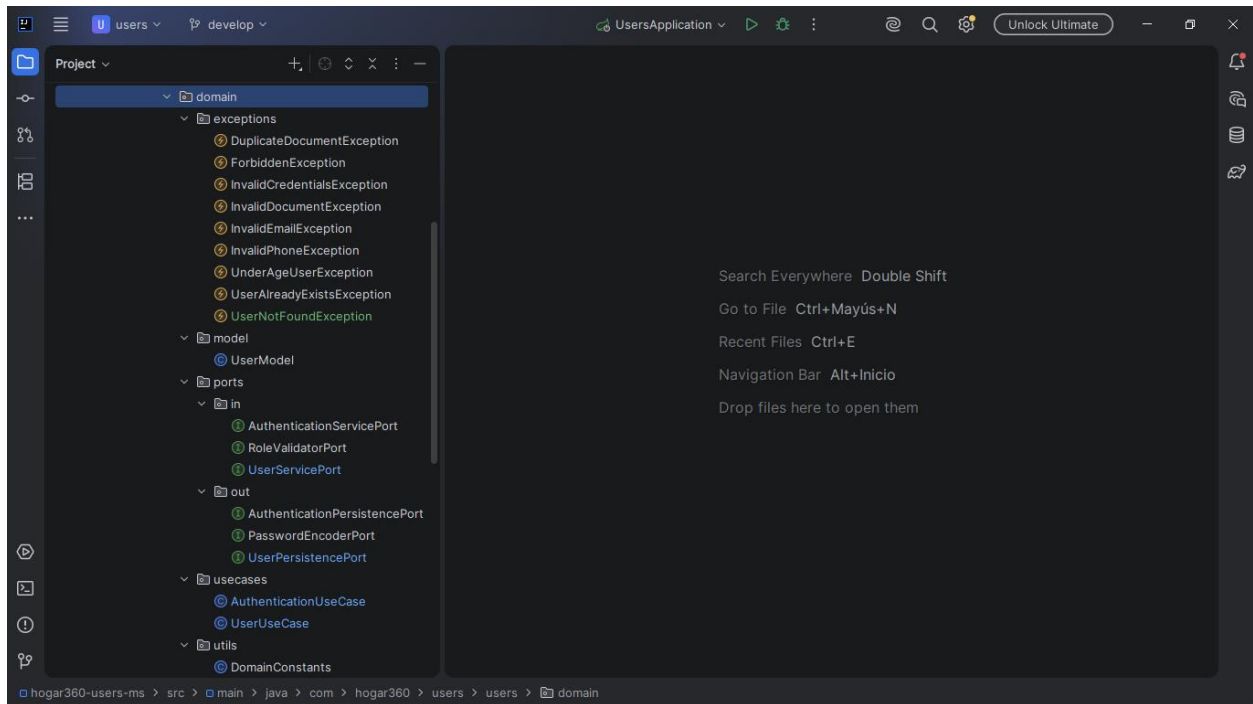
Estos modelos representan los conceptos del negocio inmobiliario y son independientes de cualquier framework o tecnología de persistencia.

El dominio se diseñó para garantizar que las reglas del negocio permanezcan desacopladas de la infraestructura.

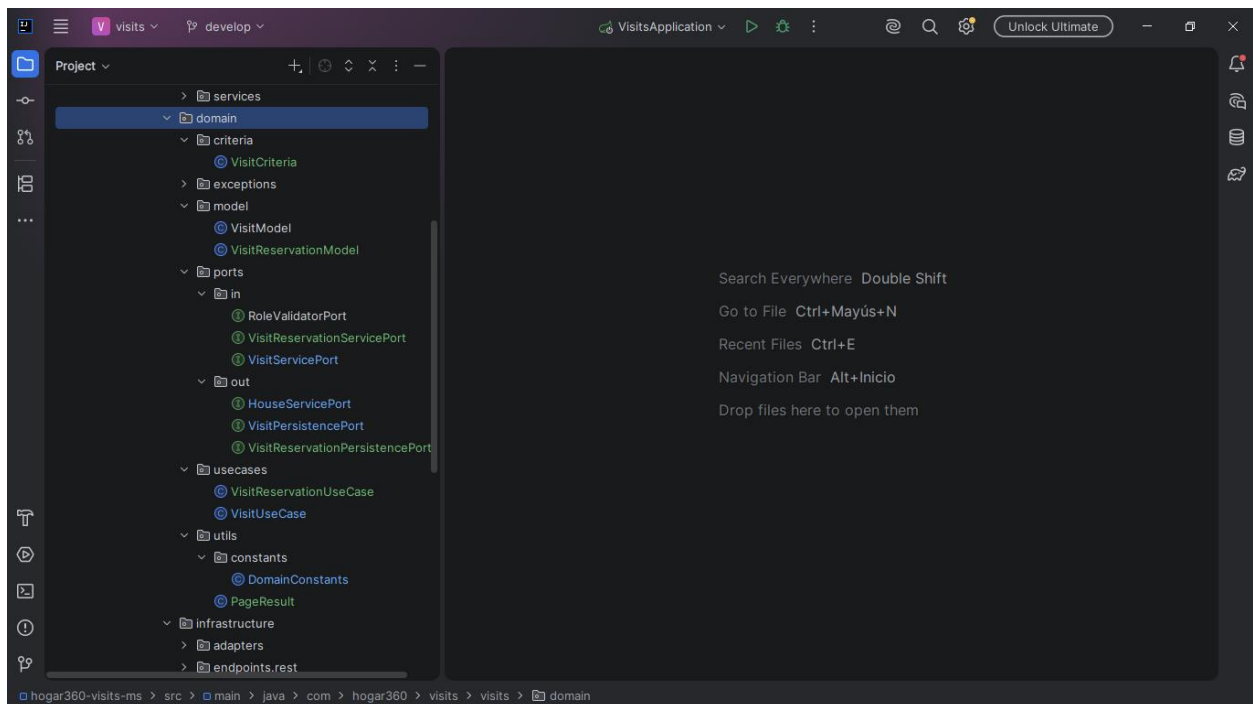
Houses microservice



Users microservice



Visits microservice



CASOS DE USO

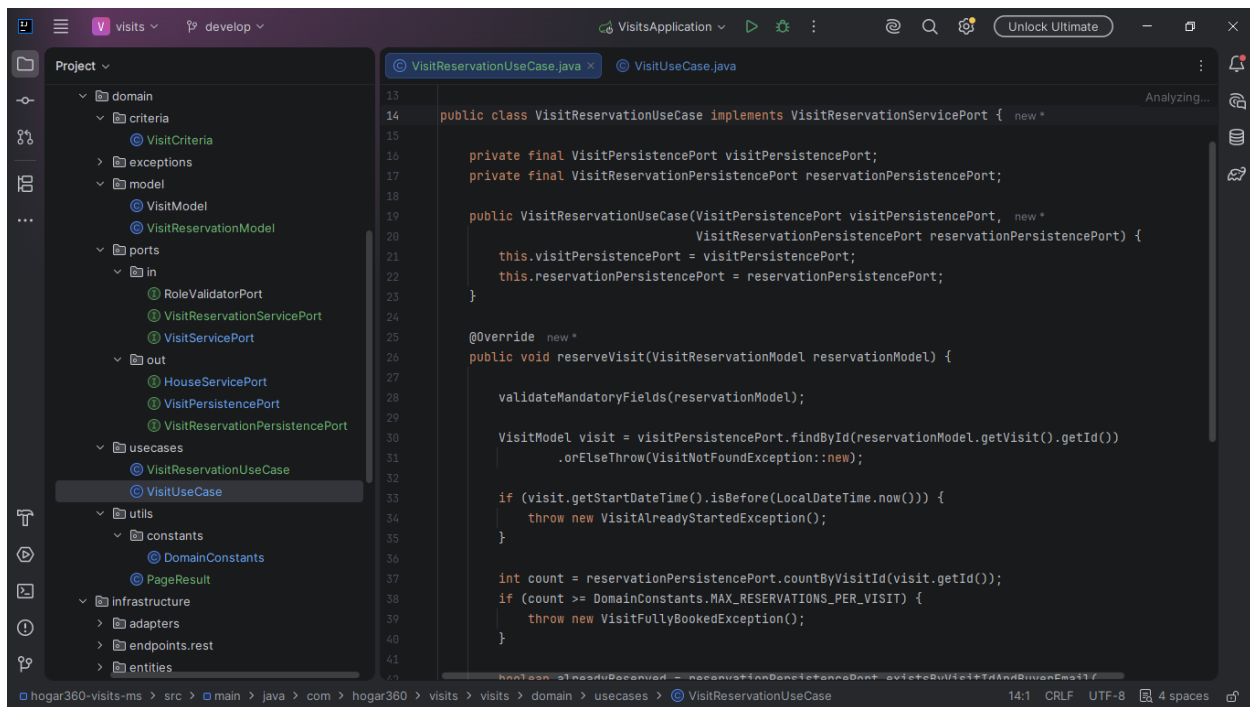
Los casos de uso concentran la lógica del negocio y coordinan la interacción entre el dominio y los puertos de persistencia.

Entre los principales casos de uso implementados se encuentran:

- HouseUseCase
- CategoryUseCase
- LocationUseCase
- VisitUseCase
- VisitReservationUseCase

Cada uno implementa las reglas de negocio necesarias para cumplir los requerimientos del proyecto.

Por ejemplo, el caso de uso de reservas verifica la disponibilidad del horario antes de registrar una nueva visita.



PUERTOS

Los puertos representan contratos entre la lógica del negocio y la infraestructura.

Se implementaron dos tipos de puertos:

Puertos de entrada

Definen las operaciones que puede ejecutar la aplicación.

Ejemplos:

- Crear casa
- Consultar categorías
- Registrar visita

Puertos de salida

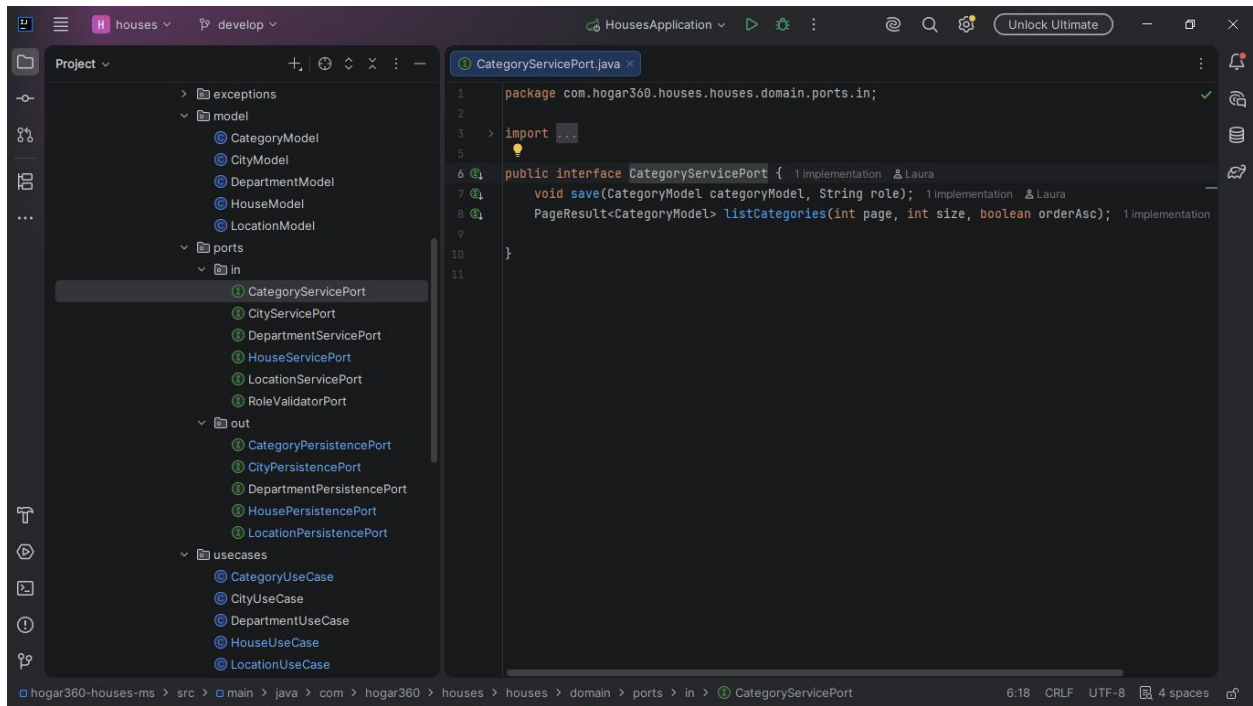
Definen las operaciones necesarias para acceder a recursos externos.

Ejemplos:

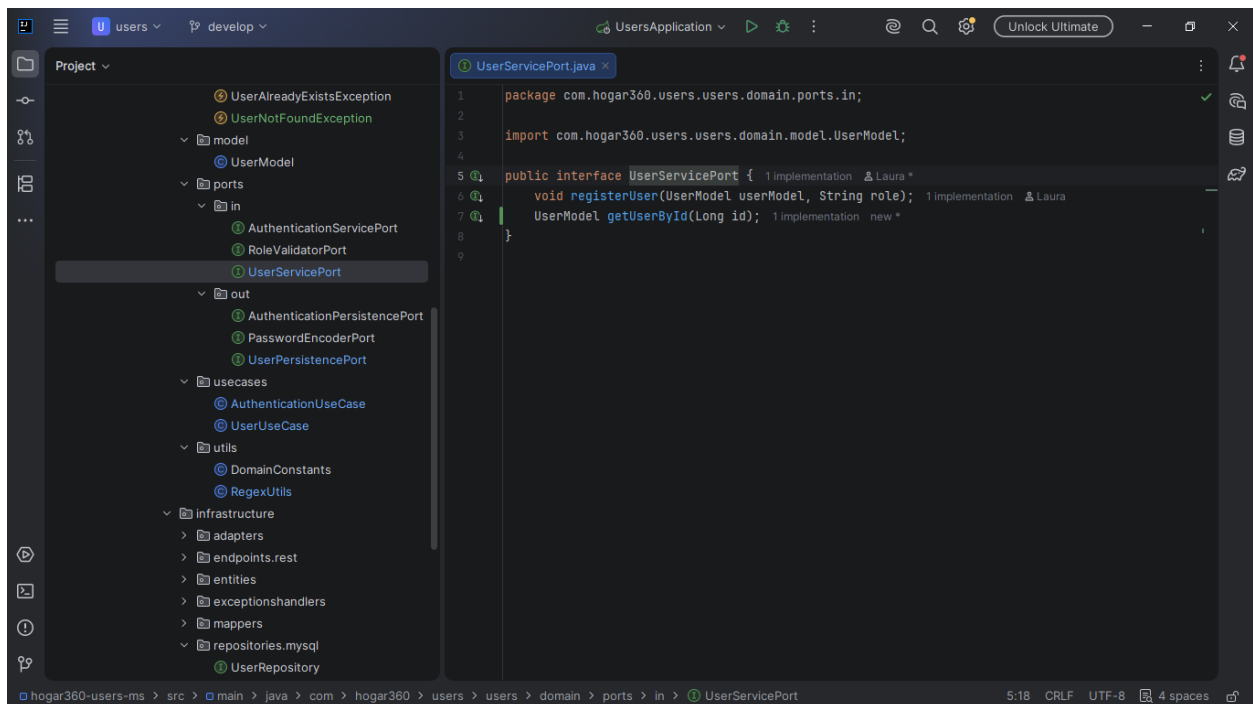
- Guardar casas
- Consultar ubicaciones
- Buscar visitas

Gracias a estos contratos, el dominio permanece independiente de Spring Boot y de JPA.

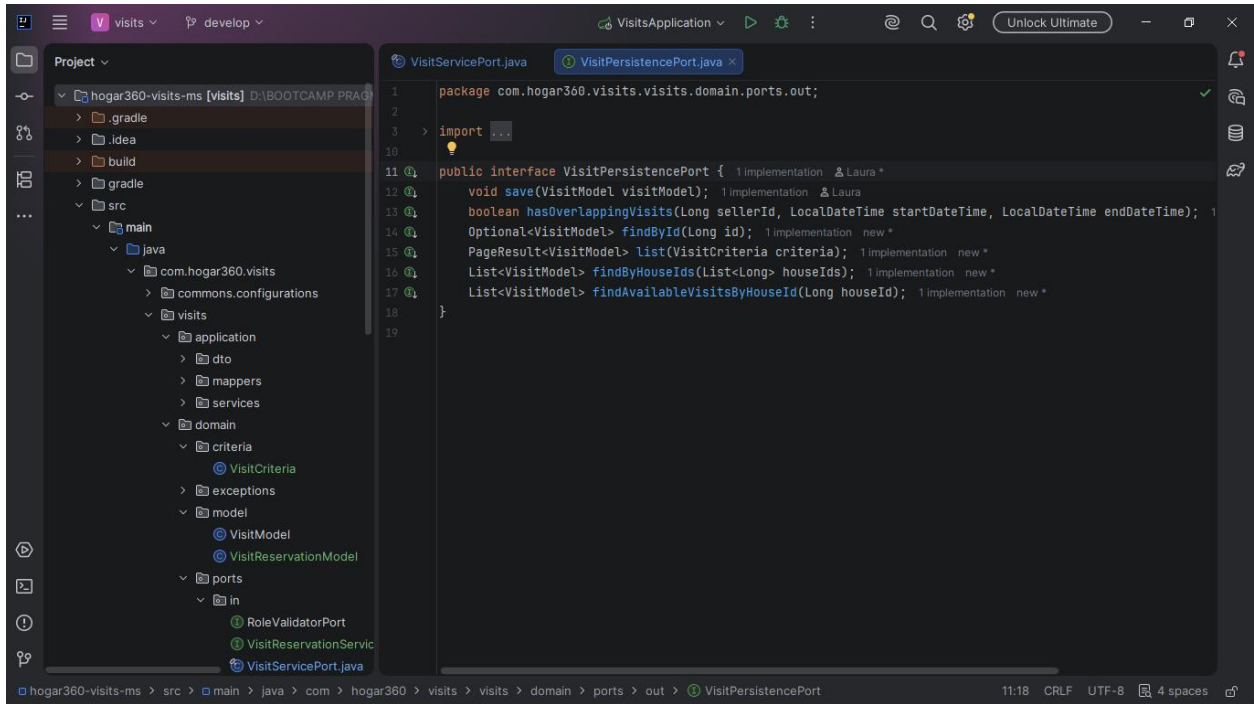
Houses ports



Users ports



Visits ports



ADAPTADORES DE PERSISTENCIA

Los adaptadores de persistencia implementan los puertos de salida utilizando Spring Data JPA.

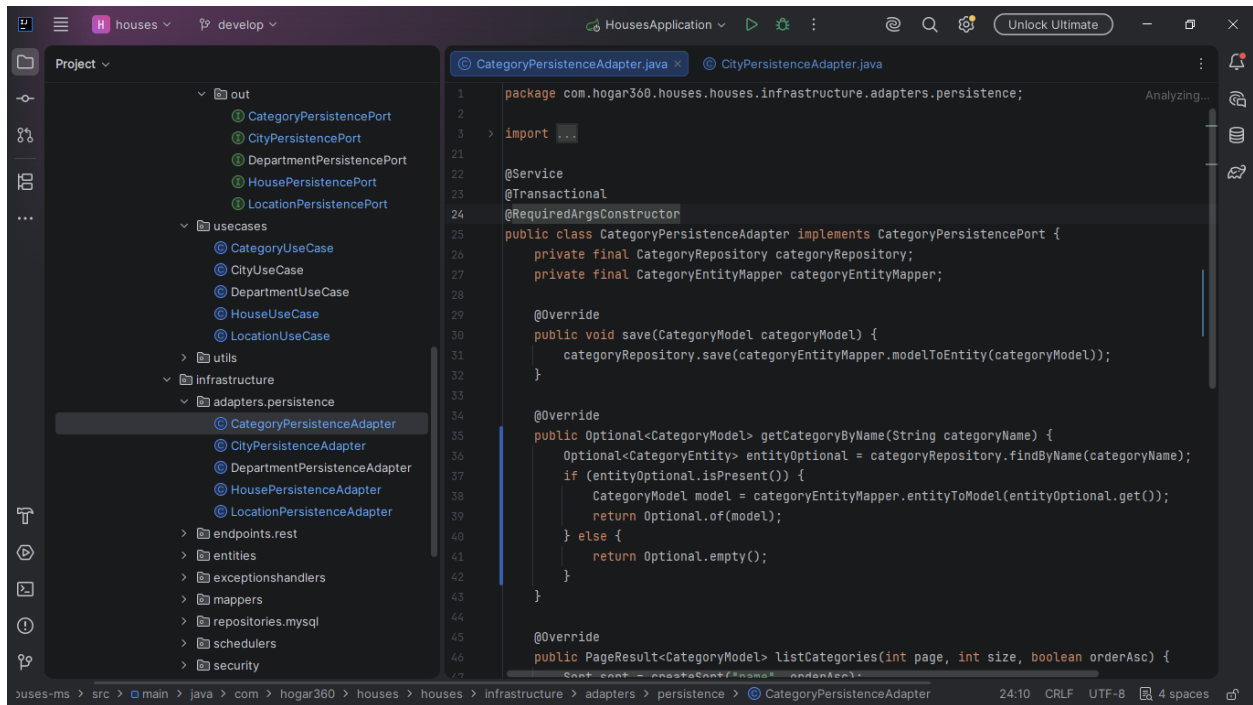
Entre sus responsabilidades se encuentran:

- Guardar información.
- Consultar registros.
- Actualizar entidades.
- Eliminar datos.

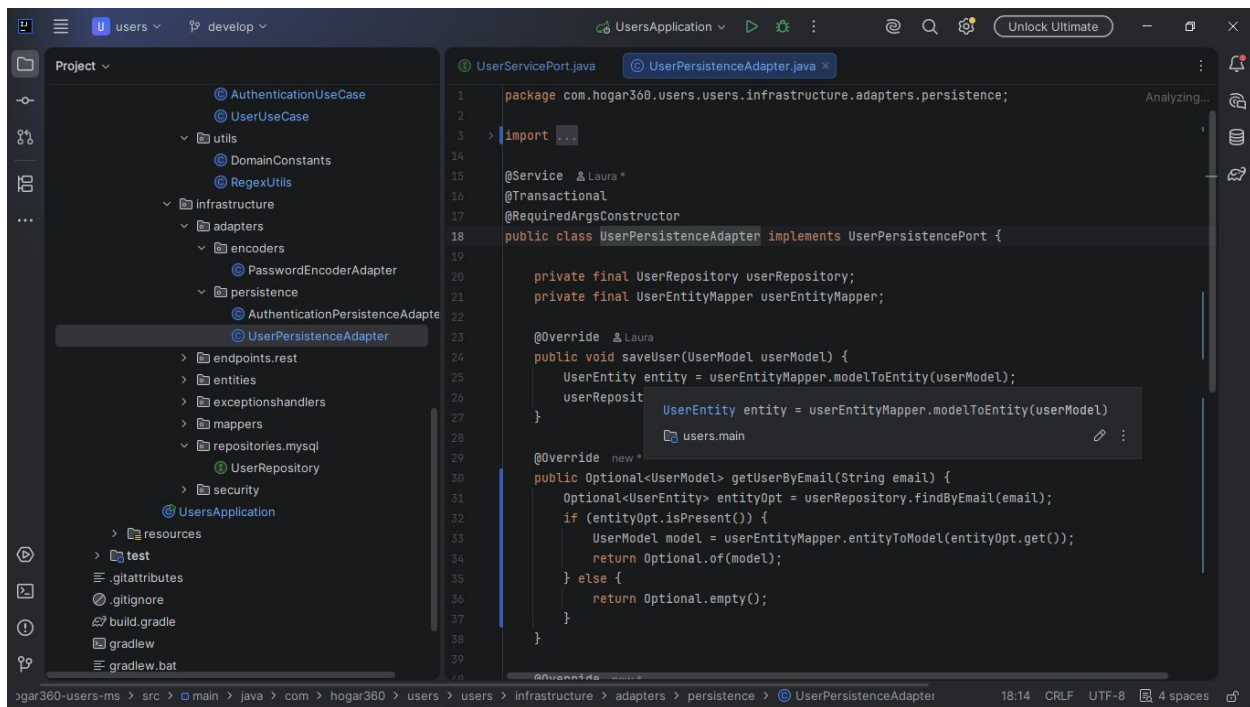
Además, utilizan MapStruct para convertir entre entidades de base de datos y modelos del dominio.

Houses

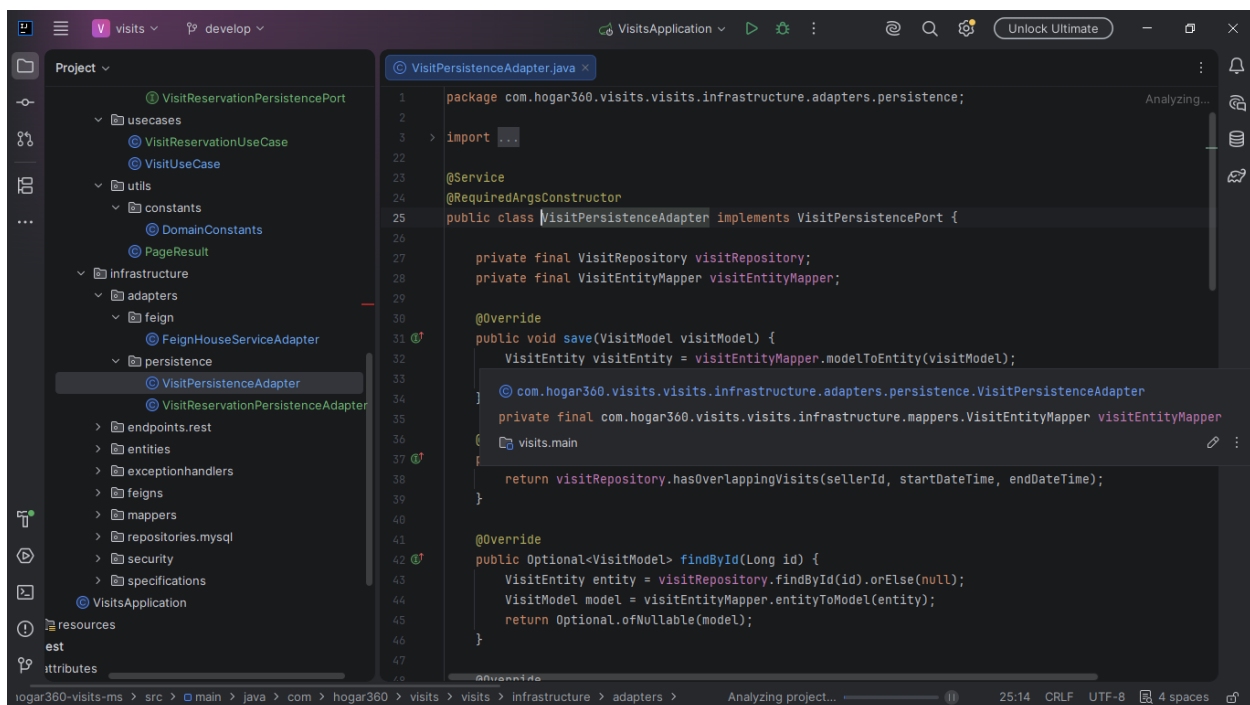
Adapters



Users Adapters



Visits Adapters



ADAPTADORES REST

Los adaptadores de entrada fueron implementados mediante controladores REST.

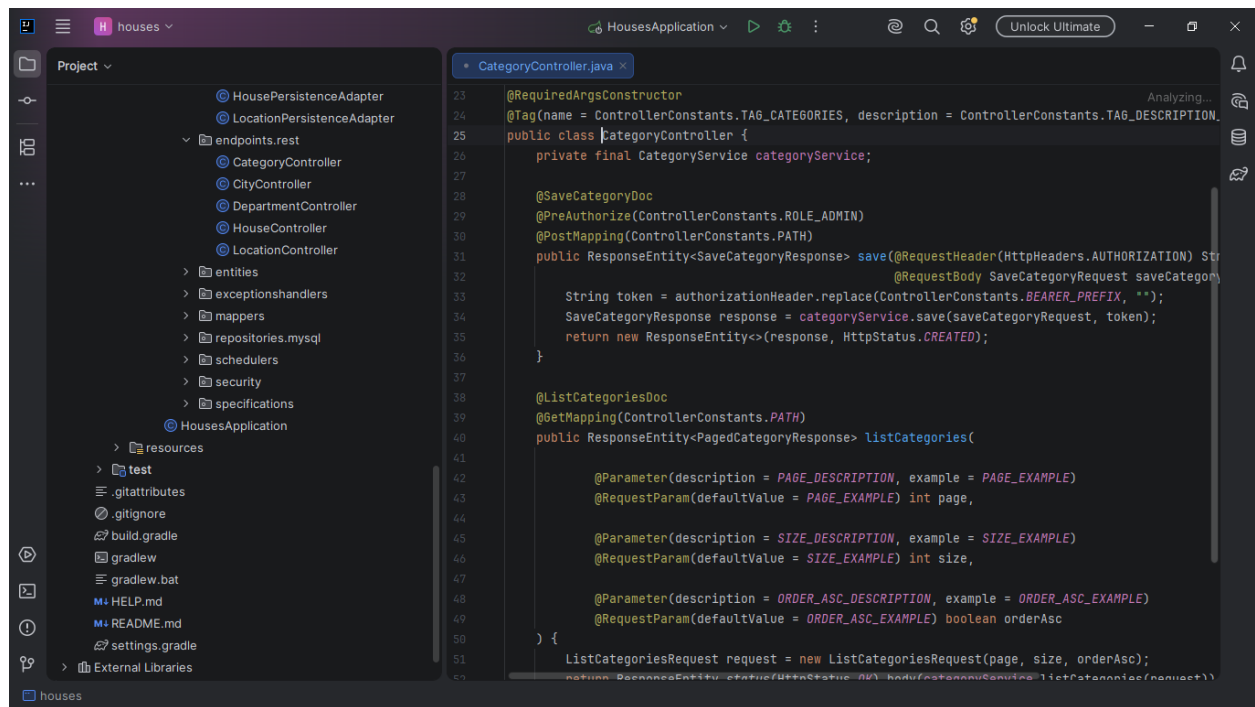
Cada controlador recibe las solicitudes HTTP provenientes del frontend y delega el procesamiento al caso de uso correspondiente.

Entre los controladores desarrollados se encuentran:

- HouseRestController
- CategoryRestController
- LocationRestController
- VisitRestController

Esta implementación mantiene separada la capa de presentación de la lógica del negocio.

Houses microservice endpoint controllers



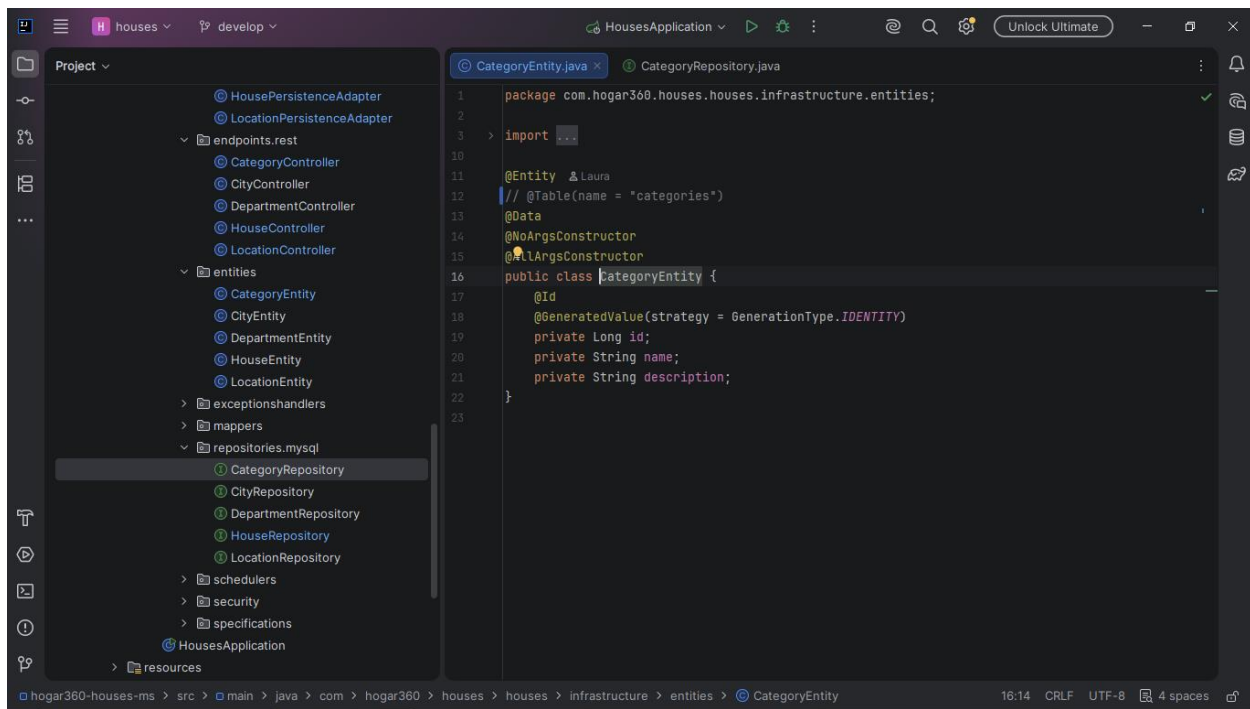
PERSISTENCIA DE DATOS

La información del sistema se almacena en una base de datos relacional mediante Spring Data JPA.

Se implementaron:

- Entidades
- Repositorios
- Mappers
- Adaptadores de persistencia

Esta organización permite mantener sincronizados los modelos del dominio con la base de datos sin acoplar ambas capas.

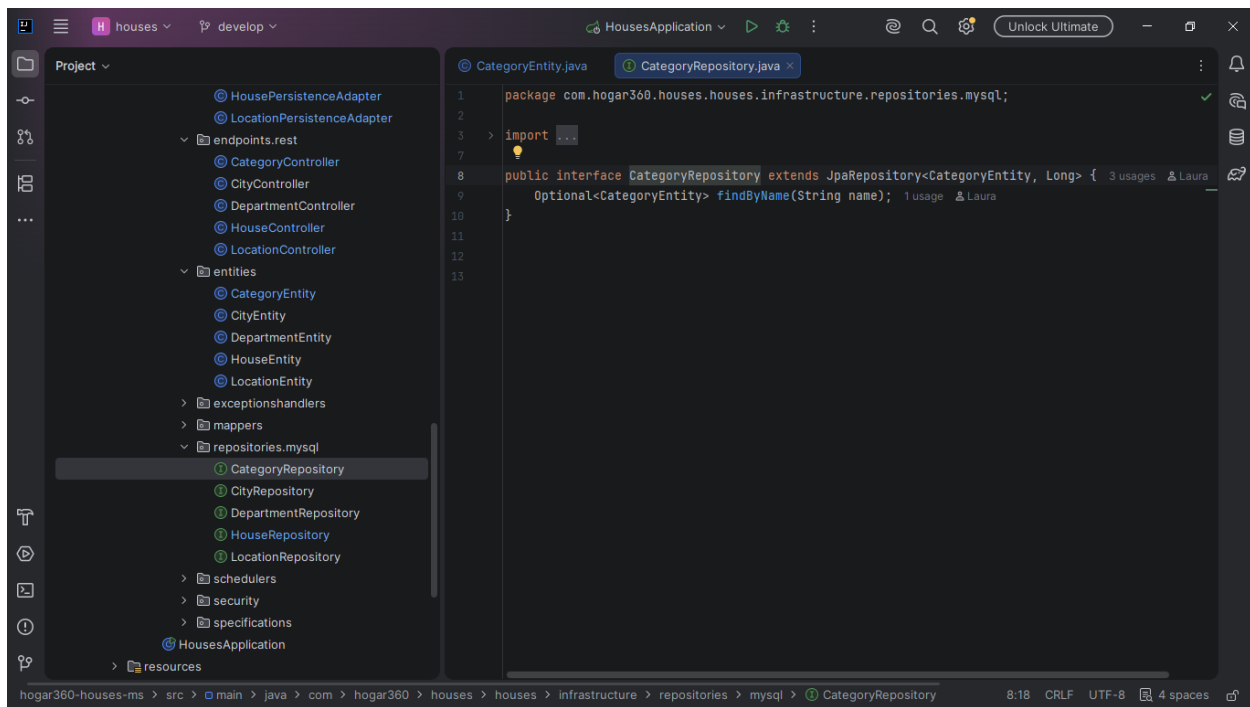


Project

- HousePersistenceAdapter
- LocationPersistenceAdapter
- endpoints.rest
 - CategoryController
 - CityController
 - DepartmentController
 - HouseController
 - LocationController
- entities
 - CategoryEntity
 - CityEntity
 - DepartmentEntity
 - HouseEntity
 - LocationEntity
- exceptionshandlers
- mappers
- repositories.mysql
 - CategoryRepository
 - CityRepository
 - DepartmentRepository
 - HouseRepository
 - LocationRepository
- schedulers
- security
- specifications
- HousesApplication
- resources

```
1 package com.hogar360.houses.houses.infrastructure.entities;
2
3 import javax.persistence.*;
4
5 @Entity
6 @Table(name = "categories")
7 @Data
8 @NoArgsConstructor
9 @AllArgsConstructor
10 public class CategoryEntity {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14     private String name;
15     private String description;
16 }
```

hogar360-houses-ms > src > main > java > com > hogar360 > houses > houses > infrastructure > entities > CategoryEntity 16:14 CRLF UTF-8 4 spaces



Project

- HousePersistenceAdapter
- LocationPersistenceAdapter
- endpoints.rest
 - CategoryController
 - CityController
 - DepartmentController
 - HouseController
 - LocationController
- entities
 - CategoryEntity
 - CityEntity
 - DepartmentEntity
 - HouseEntity
 - LocationEntity
- exceptionshandlers
- mappers
- repositories.mysql
 - CategoryRepository
 - CityRepository
 - DepartmentRepository
 - HouseRepository
 - LocationRepository
- schedulers
- security
- specifications
- HousesApplication
- resources

```
1 package com.hogar360.houses.houses.infrastructure.repositories.mysql;
2
3 import javax.persistence.*;
4
5 public interface CategoryRepository extends JpaRepository<CategoryEntity, Long> {
6     Optional<CategoryEntity> findByName(String name);
7 }
8
9
10
11
12
13
```

hogar360-houses-ms > src > main > java > com > hogar360 > houses > houses > infrastructure > repositories > mysql > CategoryRepository 8:18 CRLF UTF-8 4 spaces

MANEJO DE EXCEPCIONES

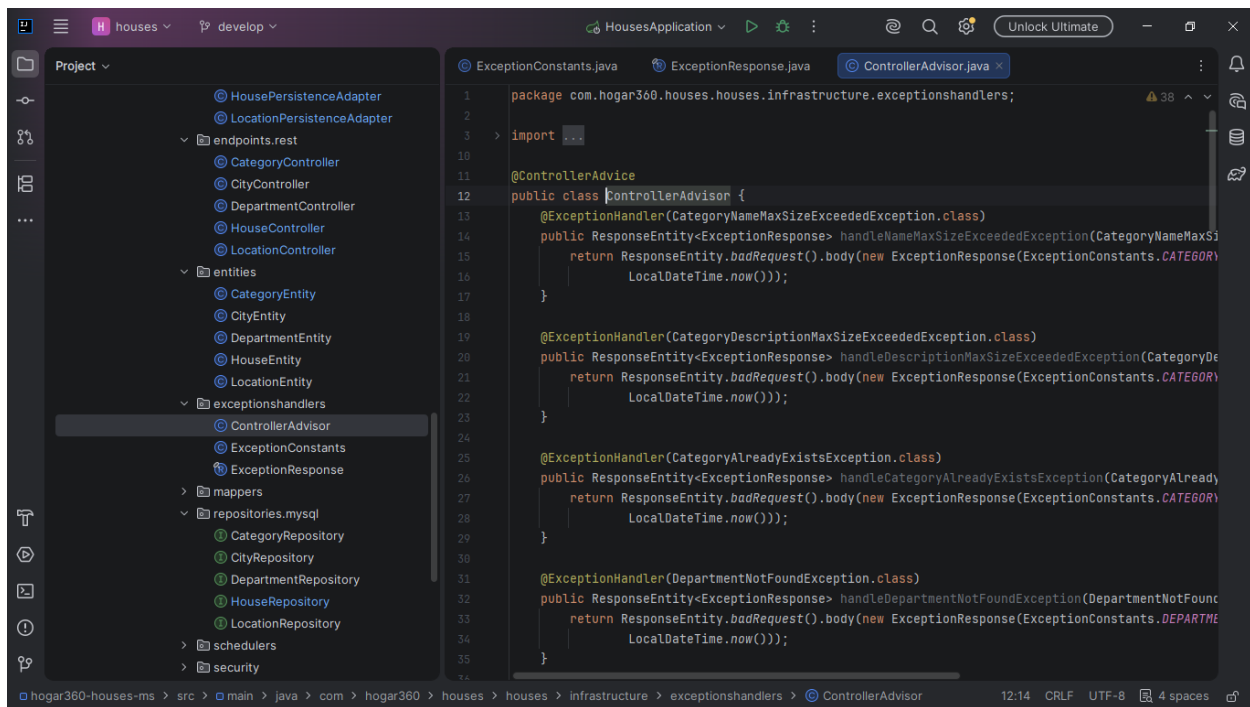
El sistema implementa un manejo centralizado de excepciones utilizando `@ControllerAdvice`.

Se desarrollaron excepciones personalizadas para representar distintos escenarios de error, permitiendo devolver respuestas HTTP claras y consistentes.

Entre los errores controlados se encuentran:

- Recursos no encontrados.
- Datos inválidos.
- Reglas de negocio incumplidas.
- Conflictos al registrar información.

Este enfoque mejora la experiencia del consumidor de la API y facilita el mantenimiento del código.



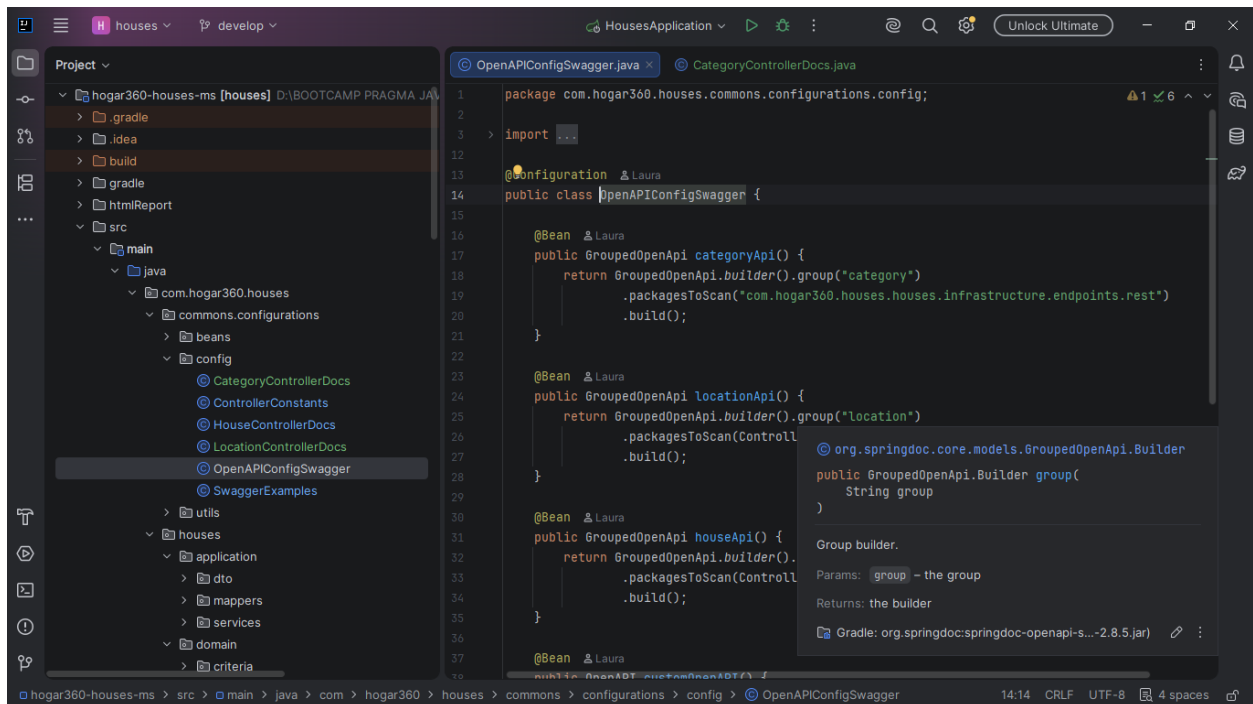
DOCUMENTACIÓN DE LA API

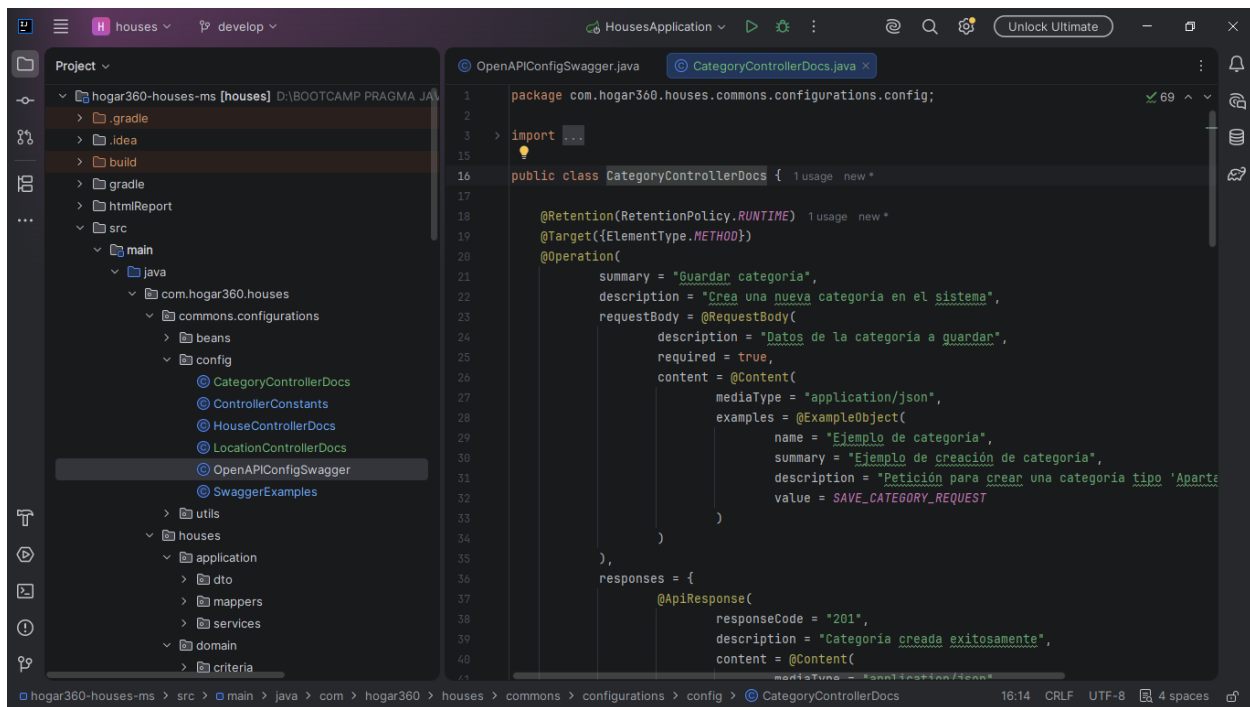
Para facilitar el consumo de los servicios REST se utilizó **Swagger/OpenAPI**, permitiendo documentar automáticamente los endpoints del sistema.

La documentación incluye:

- Métodos disponibles.
- Parámetros de entrada.
- Respuestas esperadas.
- Códigos HTTP.

Esta herramienta simplifica la integración entre el backend y el frontend.





RESULTADOS OBTENIDOS

La implementación del backend permitió desarrollar un sistema organizado y modular que responde a los requerimientos funcionales definidos para la plataforma Hogar360.

La utilización de la Arquitectura Hexagonal facilitó la separación de responsabilidades entre las distintas capas de la aplicación, mientras que la implementación de casos de uso, puertos y adaptadores permitió construir un software desacoplado y preparado para futuras ampliaciones.

Asimismo, la incorporación de validaciones, manejo de excepciones y documentación mediante Swagger contribuyó a mejorar la calidad, mantenibilidad y usabilidad de la API.

CONCLUSIÓN

El desarrollo del backend de Hogar360 permitió aplicar conocimientos relacionados con programación orientada a objetos, desarrollo de APIs REST y arquitectura de software basada en puertos y adaptadores. La organización del proyecto favoreció un código limpio, modular y fácilmente mantenible, reduciendo el acoplamiento entre la lógica del negocio y la infraestructura.

La implementación de módulos como la gestión de casas, categorías, ubicaciones y visitas demostró la importancia de estructurar el software a partir de los requerimientos funcionales, garantizando el cumplimiento de las reglas del negocio y una adecuada interacción con la base de datos. Además, el uso de herramientas como Spring Boot, Spring Data JPA, MapStruct y Swagger permitió optimizar el desarrollo y la documentación de la aplicación, obteniendo un backend robusto y preparado para integrarse con el frontend de la plataforma.